Accessing and Processing Sensitive Data in Galaxy

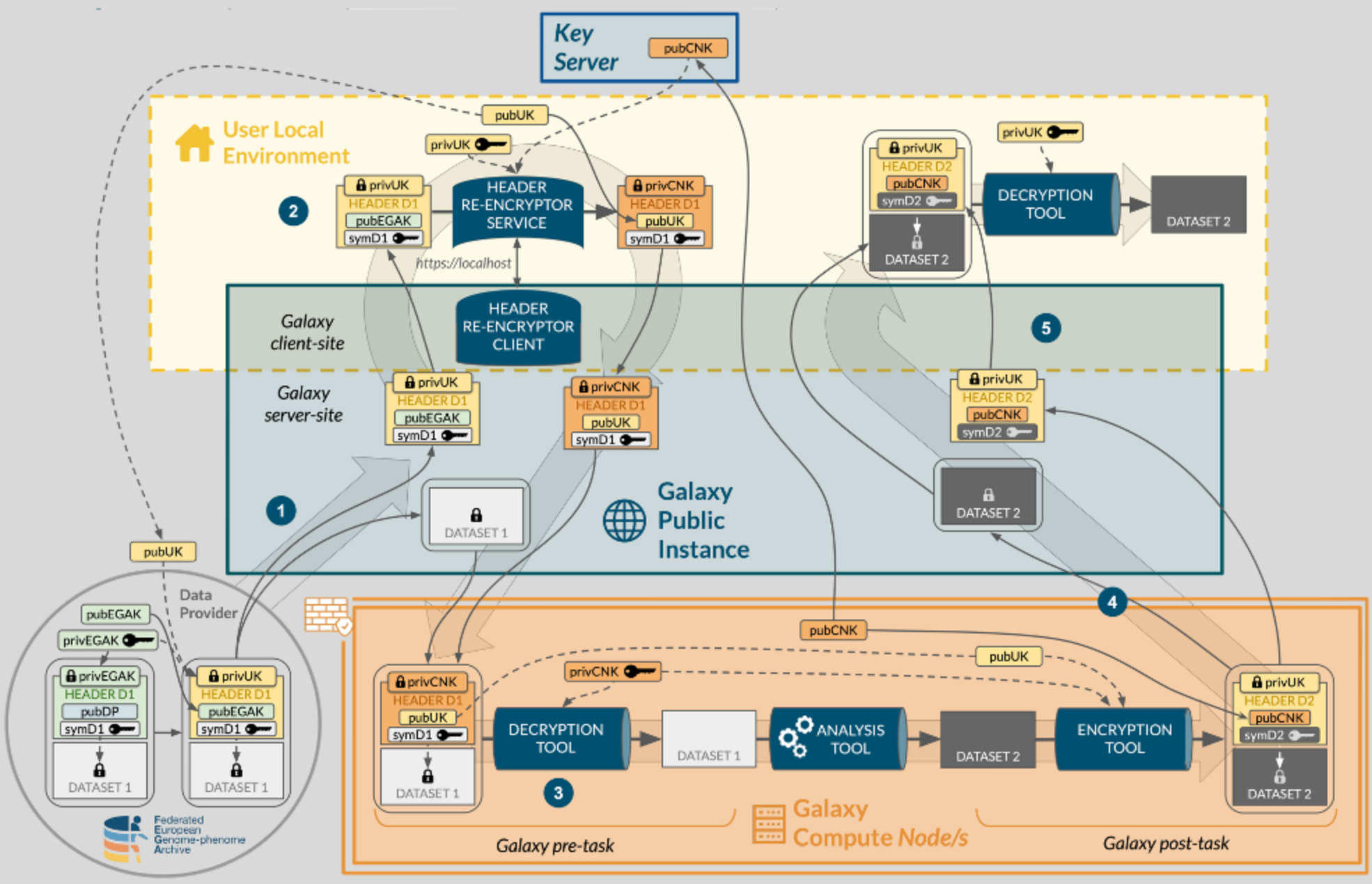
Sveinung Gundersen[1], Laia Codó[2,3], Kjell Petersen[4], José M. Fernández[2,3], María Chavero-Diez[2,3], Pável Vázquez[1], Abdulrahman Azab[1], Marco A. Tangaro[5], Krzysztof Poterlowicz[6], Frederik Coppens[9], Josep Ll. Gelpí [2,3,7], Björn Grüning[8], Salvador Capella-Gutierrez[2,3]

[1] University of Oslo (UiO), [2] Spanish National Bioinformatics Institute (INB/ELIXIR-ES); [3] Barcelona Supercomputing Center (BSC), Spain; Norway; [4] University of Bergen (UiB), Norway; [5] Italian National Research Council (CNR), Italy; [6] Computational and Data Driven Science, University of Bradford (UoB); [7] Dept Biochemistry and Molecular Biomedicine, University of Barcelona (UB); [8] University of Freiburg, Germany; [9] Data Core, VIB, Belgium

INTRODUCTION

In the context of the ELIXIR Implementation Study 'Strengthen Data Management in Galaxy' we explored novel strategies for access and processing of sensitive datasets in Galaxy in a secure and scalable manner.

To this end, we propose an approach to access, transfer and process (F)EGA Crypt4GH encrypted datasets into a secure environment using a multi-user public Galaxy instance as the enactor and controller of the full process.



Note: the figure is explained in detail in the next slides...

Contact

Sveinung Gundersen
sveinugu@ifi.uio.no

inB
BSC Barcelona Supercomputing Center
UNIVERSITY OF OSLO
UNIVERSITY OF BERGEN
BY-COVID
universität freiburg
VIB
EUCAN IMAGE
elixir
@ELIXIREurope /company/elixir-europe

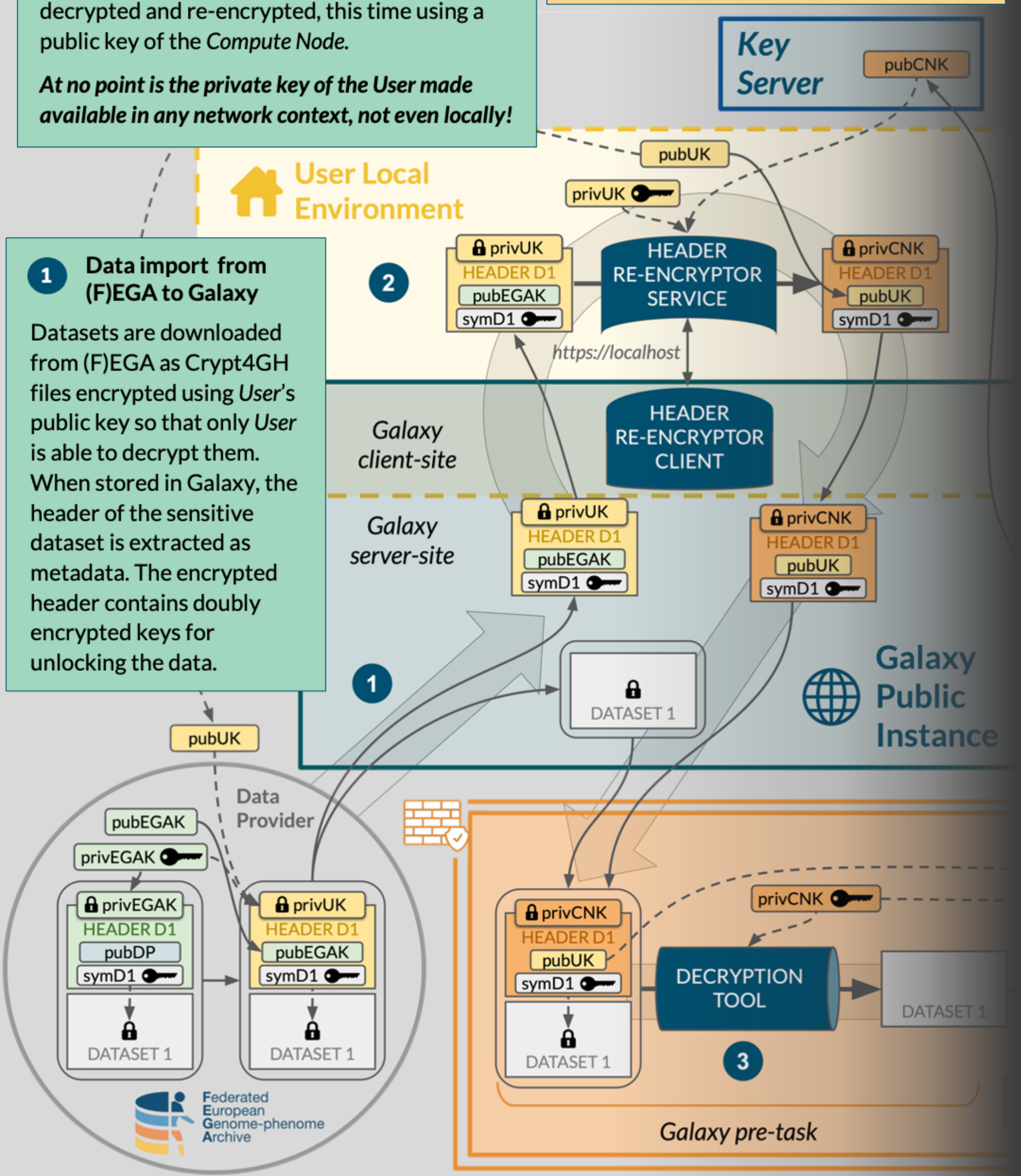
Accessing and Processing Sensitive Data in Galaxy

2 Crypt4GH header re-encryption

To enable data decryption in Galaxy compute nodes, the original header is sent to a REST server in the *User*'s local machine. The header is decrypted and re-encrypted, this time using a public key of the *Compute Node*.

At no point is the private key of the User made available in any network context, not even locally!

A *Key Server* will handle provisioning and renewal of public keys of the *Compute Nodes*. For security, the keys should be short-lived and possibly tailored for specific users.



1 Data import from (F)EGA to Galaxy

Datasets are downloaded from (F)EGA as Crypt4GH files encrypted using *User*'s public key so that only *User* is able to decrypt them. When stored in Galaxy, the header of the sensitive dataset is extracted as metadata. The encrypted header contains doubly encrypted keys for unlocking the data.

Legend

pubX: public key for X
privX: private key for X
EGAK: internal keys for (F)*EGA service*
UK: *User*'s Keys
CNK: *Compute Node*'s Keys
symDY: symmetric key for *Dataset Y*

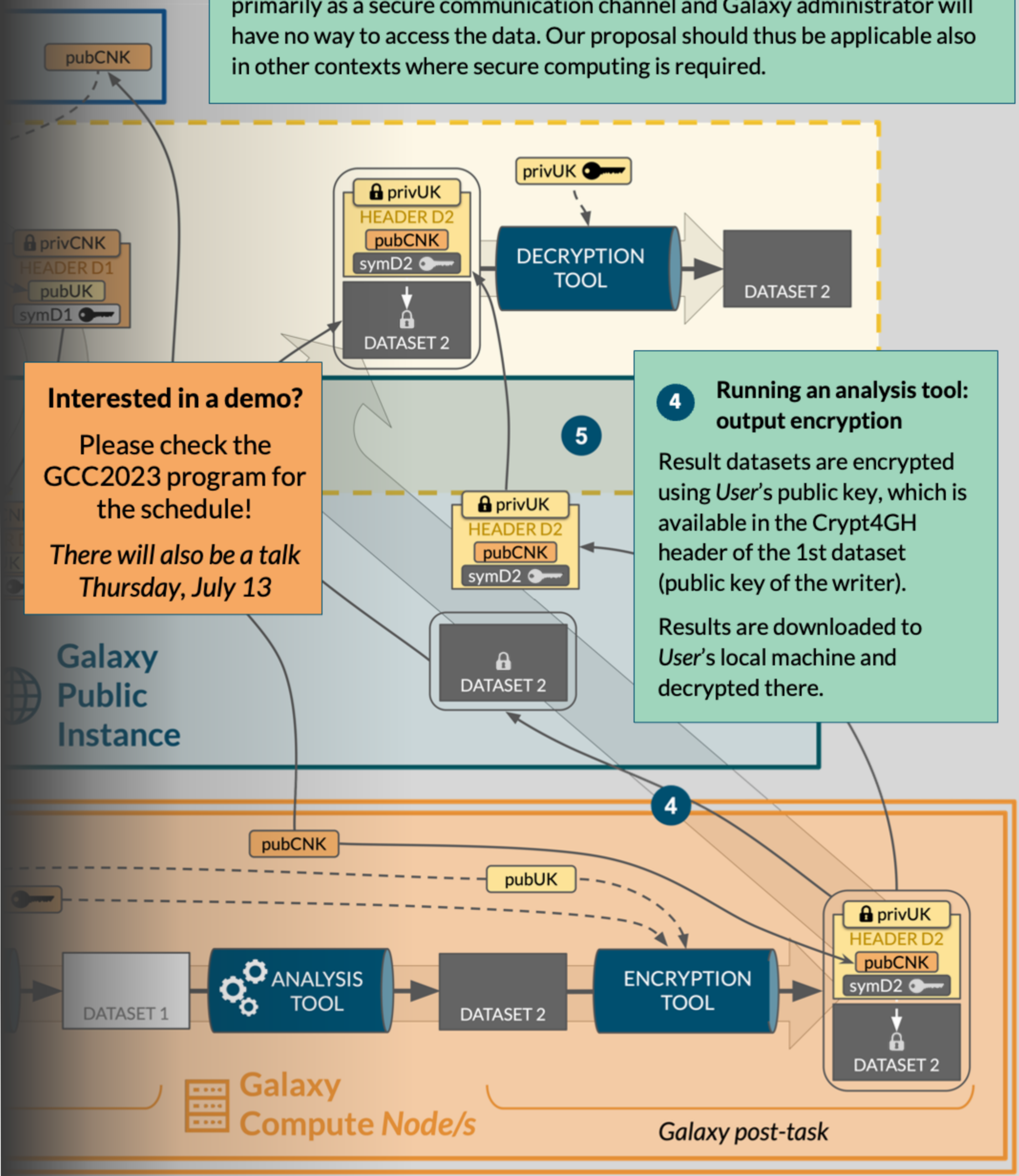
3 Running an analysis tool: input decryption

Before running an analysis tool or workflow, input data is decrypted on the compute node, directly before being processed. Analysis tools remain unaltered.

Accessing and Processing Sensitive Data in Galaxy

5 Galaxy server as a secure communication channel

Our suggestion ensures that data is always kept in encrypted form when in Galaxy and the Crypt4GH header is at all times encrypted for exclusive access by either the *User* or the *Compute Node*. Galaxy in essence functions primarily as a secure communication channel and Galaxy administrator will have no way to access the data. Our proposal should thus be applicable also in other contexts where secure computing is required.



Interested in a demo?

Please check the GCC2023 program for the schedule!

There will also be a talk Thursday, July 13

4 Running an analysis tool: output encryption

Result datasets are encrypted using *User*'s public key, which is available in the Crypt4GH header of the 1st dataset (public key of the writer).

Results are downloaded to *User*'s local machine and decrypted there.

A general approach to support sensitive data analysis in *Compute Nodes* could be to extend the common template launching jobs to Galaxy runners to include instructions on how to encrypt/decrypt data in the job destination.

Legend

pubX: public key for X
privX: private key for X
EGAK: internal keys for (F)*EGA service*
UK: *User*'s Keys
CNK: *Compute Node*'s Keys
symDY: symmetric key for *Dataset Y*